



Personalised Code and Error Predictions in Programming Education via Large Language Models

Martha Shaka^(✉), Diego Carraro, and Kenneth N. Brown

School of Computer Science and IT, University College Cork, Cork, Ireland
`m.shaka@cs.ie`, `k.brown@cs.ucc.ie`

Abstract. Novice programmers frequently struggle with syntax errors, which hinder learning progression and motivation. While AI-based programming tools (AIPTs) provide automated feedback, most lack personalisation and proactive interventions. Off-the-shelf Large Language Models (LLMs) are primarily trained to generate correct solutions, not to simulate students' mistakes. Knowledge Tracing (KT) models estimate student mastery over time and predict future performance. Still, traditional KT methods focus on binary pass/fail predictions, limiting their ability to diagnose misunderstandings and provide actionable feedback. This study integrates fine-tuned LLMs, Codellama-13B and Mistral-7B, with KT to identify student-specific gaps by predicting likely syntax errors. Our framework introduces an intermediate step of personalised code generation, where the model simulates how a student might attempt a task. The generated code is then compiled to extract errors aligned with the student's historical misconceptions. We refine code alignment and error prediction accuracy using Supervised Fine-Tuning and Reinforcement Learning (ORPO). Experimental results on a real-world Java dataset show a +34% improvement in error prediction and +27% increase in code similarity over baseline models. These findings lay a foundation for proactive, personalised learning support. Future work will focus on integrating this framework into AIPTs to recommend adaptive exercises, materials, and hints before students attempt a task.

Keywords: LLMs · Personalised Code Generation · Knowledge Tracing

1 Introduction

Novice programmers often struggle to master foundational programming concepts due to difficulties understanding syntax rules, logical structures, and debugging processes [1]. These challenges, compounded by unfamiliar terminology and the complexities of programming language rules, lead to frequent errors,

particularly in introductory programming courses. Syntax errors, such as missing semicolons or brackets, are widespread in early-stage learning, often overshadowing deeper conceptual issues and hindering students' ability to develop problem-solving skills [14]. Addressing these challenges requires personalised support that adapts to each student's specific learning gaps, a task that traditional classroom instruction struggles to scale.

AI-based programming tools (AIPs) have improved support for novice programmers, but most still provide generic or post-submission feedback, limiting their capacity for proactive personalisation. Recent LLM-based systems, such as GPT-4-driven tools [10], show promise by tailoring feedback based on student history and performance. However, these remain largely reactive, lacking the ability to offer preparatory exercises to reinforce knowledge gaps or delivering targeted hints aligned with students' expected problem-solving approaches before they attempt tasks.

Knowledge Tracing (KT) is a key technology in educational data mining, offering a framework for estimating students' mastery over time, predicting performance, and identifying knowledge gaps [19]. KT-powered tools like PyFiXv [18] leverage pass/fail predictions to link students' current and prior submissions, providing insights into their learning progression and enabling possibilities of tailored tuition. However, traditional KT models are limited in scope, as their reliance on binary pass/fail predictions restricts the granularity of feedback. This limitation makes it difficult to pinpoint specific areas where students struggle, such as syntax rules or logical reasoning, which are crucial for developing personalised interventions. Recent advances in KT, such as OKT [13], have demonstrated the potential of integrating LLMs to generate code personalised to students based on their learning behaviour and knowledge modelling. While promising, this approach focuses on logical errors and relies on embedding-based student histories, overlooking syntactical errors and the advantages of instruction-tuned LLMs. Moreover, they lack evaluations of whether the generated solutions align with actual student errors, limiting their practical application in education.

We propose a framework that uses an LLM to generate code similar to that expected from a given student. We then pass the code through a compiler to receive specific syntactical error reports and predict errors the student will likely make. Our framework enhances program synthesis by integrating a student-specific knowledge component based on a trace of their historical submissions and error reports. These elements are presented in a structured natural language format designed to align with the strengths of LLMs in processing and understanding instructional data [17]. Our methodology combines the strengths of pre-trained LLMs, instruction-supervised fine-tuning (SFT), and reinforcement learning (RL). Initially, we evaluate pre-trained LLMs in a zero-shot setting, using in-context inference to generate code without additional training. While these models show some capability, their outputs often fail to align with individual student knowledge or reflect the specific errors novices are likely to make. To overcome this, we apply SFT to tailor the models for novice programming behaviours and their learning progression. Finally, RL further refines the models,

enabling them to generate code that aligns with expected student errors. The source code is available at [our GitHub repository](#).

Evaluations on a real-world Java dataset show that our reinforcement learning approach, particularly using ORPO, improves code similarity by +27% and error prediction accuracy by +34% compared to baseline models like OKT. These findings highlight the model’s capability to align more closely with the specific syntax errors students are likely to make, offering valuable insights into their learning needs. By predicting likely mistakes and plausible code solutions, educators and AIPs can proactively recommend tailored exercises to address identified weaknesses or craft adaptive hints aligned with a student’s expected problem-solving approach. This framework shifts the focus from reactive error correction to proactive support, offering transformative potential in how novice programmers engage with and learn programming concepts.

The rest of this paper is structured as follows: Sect. 2 reviews related work on KT and LLM-based code generation. Section 3 outlines our task formulation, dataset, and fine-tuning strategies. Section 4 describes the experimental setup, and Sect. 5 presents key findings. We conclude in Sect. 6 with a discussion of implications and future directions.

2 Related Work

Knowledge Tracing (KT) is a core technique in educational data mining that models a student’s evolving knowledge state to predict future performance [19]. Traditional KT models, such as Bayesian Knowledge Tracing (BKT) [2], predict binary outcomes (e.g., pass/fail) based on a student’s estimated mastery of knowledge components. BKT has been applied in programming education for performance prediction [28] and exercise recommendations [15]. Logistic KT models extend this by incorporating task difficulty and practice frequency to improve prediction accuracy [21], but these models struggle to capture long-term dependencies and the sequential nature of learning [19]. To address these limitations, deep learning-based approaches like Deep Knowledge Tracing (DKT) [19] use recurrent neural networks to dynamically track knowledge states. Extensions such as Code-DKT [24] and HELP-DKT [11] enhance prediction in programming contexts by incorporating code-specific features. Pre-trained models like CodeBERT have also been fine-tuned on submission-level data to provide more personalised feedback [10].

Building on these developments, recent work has expanded KT beyond binary prediction. Error-DKT [22] predicts common syntax errors, enabling more precise diagnostic feedback. Generative approaches such as OKT [13] integrate DKT embeddings and task contexts to generate plausible code aligned with a student’s learning state. TICTOK [3] further enhances this by evaluating whether student-passed test cases align with those passed by the generated code. However, key limitations remain. These models predominantly target logical errors, overlooking syntactical issues that are especially common among novice programmers. Moreover, they rely on embedding-based histories and do not leverage

instruction-tuned LLMs that can process student histories in natural language [8]. Lastly, while TICTOK introduces test case alignment, existing models lack evaluations of whether generated code truly reflects student-specific misconceptions, an essential aspect for delivering effective, personalised support.

In parallel, advances in pre-trained programming models such as CodeT5, CodeLlama, and StarCoder have shown success in general code generation tasks. Yet, they often produce non-compilable or functionally incorrect code [25]. To improve code reliability, recent research has explored reinforcement learning (RL) approaches that optimise compilability and test case conformity. For instance, CodeRL [9] uses test feedback in an actor-critic framework, PPOCoder [25] applies Proximal Policy Optimization with functional test rewards, and PanGu-Coder2 [23] fine-tunes models based on ranked test outcomes and human preferences. However, these methods prioritise correctness and robustness over educational objectives and do not generate student-like errors or adapt outputs to a learner’s knowledge state—limiting their utility in personalised programming education.

3 Methodology

We introduce a framework that leverages generative LLMs to predict student-specific syntactical errors and support personalised programming education. The core task is divided into two steps: personalised code generation and error prediction via compilation. Unlike traditional methods that rely on pass/fail predictions or static embeddings, this approach generates plausible student-like code solutions based on behavioural traces such as past submissions and error reports. The model simulates how students might approach new problems or refine their solutions after initial attempts, aiming to predict errors specific to their knowledge and misconceptions rather than errors unrelated to their learning trajectory. We use Supervised fine-tuning (SFT) to align the model with knowledge-aware code generation, and Reinforcement Learning (RL) to refine outputs to better reflect student knowledge and observed error patterns.

3.1 Task Definition

Our approach uses a student’s submission history, provided as a contextual sequence of behavioural traces, to predict the specific errors they are likely to make in future attempts. Given a sequence of T submissions, $S_T = \{x_1, x_2, x_3, \dots, x_T\}$, where each submission x_t includes the programming problem (p_t), the submitted code (c_t), and the errors (e_t), our primary goal is to predict the specific errors (e_{T+1}) that arise in the next submission (x_{T+1}) for a new problem (p_{T+1}).

To achieve this, the framework introduces personalised code generation, where an LLM generates plausible code solutions (c_{T+1}) aligned to the student’s knowledge and misconceptions. The generated code is then compiled to predict the syntax errors (e_{T+1}) that the student would make.

3.2 Prompt Design for Fine-Tuning

The prompt (input) consists of two primary components: the system prompt and the input text. The system prompt defines the LLM’s global role, instructing it as a knowledge-tracing expert and coding assistant. The input text provides task-specific context, including a structured summary of the student’s historical performance (e.g., previous errors, final attempts, and learning trajectory). Additionally, it specifies the current programming task, outlining the expected solution format and any relevant constraints. The output contains the target code, representing the ground truth solution provided by the student and serves as a reference for evaluating or fine-tuning the model’s outputs.

3.3 Fine-Tuning Approach

We employed two fine-tuning approaches to adapt pre-trained language models for the knowledge tracing task: Supervised Fine-Tuning (SFT) and Reinforcement Learning (RL). We implemented two variations for RL: one incorporating a reward system and another without rewards.

Instruction Supervised Fine-Tuning (SFT). Instruction SFT involves fine-tuning pre-trained LLMs using structured datasets of natural language instructions paired with their corresponding outputs [17, 29]. The goal is to improve the model’s ability to effectively process and follow diverse instructions, aligning its behaviour with user objectives and domain-specific requirements. In our work, the instructions are derived from natural language prompts that include the student’s history and task description. The output is the corresponding ground truth code solution. SFT facilitates task-specific adaptation efficiently without requiring extensive retraining or architectural modifications. Also, instruction SFT addresses a key limitation of existing generative systems like OKT, where student historical information is represented by DKT embeddings rather than directly accessible in a natural language format.

Reinforcement Learning with a Reward System. To further align the generated code with a student’s competency and programming style, we employed RL-based fine-tuning methods. RL is a state-of-the-art approach in LLMs alignment with user preferences [12, 17, 23]. We use a Proximal Policy Optimization (PPO) framework to fine-tune the model, as illustrated in Fig. 1. This process involves two key components: the Policy LLM, which is the fine-tuned model, and the Reference LLM, which serves as a baseline model that isn’t fine-tuned. The Reference LLM ensures that the Policy LLM’s generation does not diverge too much from the generation that one would get from the same non-fine-tuned model. Both models are fed with the same input (student’s history, task description, committed errors, and previous code), and their outputs are compared using KL divergence. This measure acts as a regularisation mechanism during the fine-tuning process. A reward system then evaluates the Policy LLM’s outputs and provides a feedback signal to guide updates in the PPO framework.

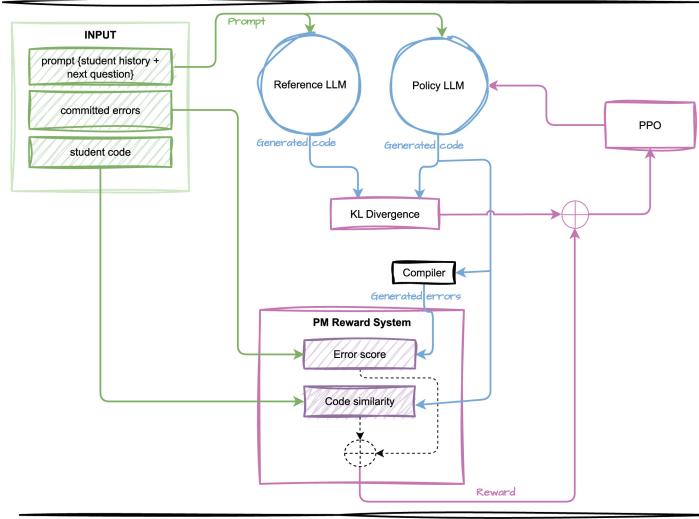


Fig. 1. Illustration of the PPO framework for aligning the Policy LLM with student-specific behaviours. The input consists of the prompt (student history and next question), committed errors, and student code. The Reference LLM acts as a baseline, while the Policy LLM generates fine-tuned outputs. KL divergence ensures the output generated by the Policy LLM remains close to that of the Reference LLM. The PM Reward System evaluates the Policy LLM’s output using two metrics: error score (from the compiler) and code similarity (CodeBLEU), combining them into a reward signal for PPO optimisation

Pattern Match (PM) Reward System

The proposed PM Reward System evaluates the Policy LLM’s output using two key components: **error score** and **code similarity**, ensuring alignment with functional correctness and stylistic requirements.

Error Score: The error score $R_e(\hat{y}, y_{\text{true}})$ is a custom-designed metric that compares compiler feedback from the generated code $\hat{y}$ against the ground truth code y_{true} . Let $f(c)$ represent the compiler’s output for code c . The error score is defined as:

$$R_e(\hat{y}, y_{\text{true}}) = \begin{cases} +F_1, & \text{if } f(\hat{y}) \text{ matches the error list from } f(y_{\text{true}}), \\ -1, & \text{if } f(\hat{y}) \text{ compiles when } f(y_{\text{true}}) \text{ generates an error,} \\ -1, & \text{if } f(\hat{y}) \text{ generates an error when } f(y_{\text{true}}) \text{ compiles.} \end{cases}$$

Here, F_1 represents the F1 score computed between the generated and ground truth error lists, encouraging the model to capture student-specific error patterns accurately.

Code Similarity: Code similarity $R_s(\hat{y}, y_{\text{true}})$ measures the alignment between the generated code $\hat{y}$ and the ground truth code y_{true} using the CodeBLEU

metric [20]. CodeBLEU evaluates multiple aspects of similarity, including n-gram overlap, syntax matching, and data flow consistency:

$$R_s(\hat{y}, y_{\text{true}}) = \text{CodeBLEU}(\hat{y}, y_{\text{true}}).$$

Combined Reward Signal: The overall reward signal $R(\hat{y}, y_{\text{true}})$ combines the error score and code similarity, balancing functional correctness with stylistic alignment:

$$R(\hat{y}, y_{\text{true}}) = \alpha R_e(\hat{y}, y_{\text{true}}) + \beta R_s(\hat{y}, y_{\text{true}}),$$

where α and β control the balance between functional correctness and stylistic alignment.

Reinforcement Learning Without a Reward System. While reward-based RL frameworks like PPO are effective for fine-tuning, they present challenges such as reward hacking, where the model exploits flaws in the reward function to maximise scores without genuinely solving the task [7]. Additionally, these methods often suffer from computational inefficiency due to their iterative nature and reliance on a reference LLM for comparison. To address these challenges, we employ Odds Ratio Preference Optimization (ORPO) [5], a monolithic alignment method that integrates preference alignment directly into the SFT training process, eliminating the need for separate RL phases. ORPO introduces an odds ratio term to distinguish between preferred (chosen) and less preferred (rejected) generation styles during SFT. During training, the LLM generates responses $\hat{y}$ based on the input x . These responses are compared against the “ground truth” or preferred response y_{chosen} as part of the SFT process. Traditionally, SFT focuses on aligning the model’s $\hat{y}$ with y_{chosen} using a loss function like Negative Log-Likelihood (NLL). The ORPO method enhances this by integrating an additional odds ratio term into the loss function. This term ensures that the model not only produces the correct response but also consistently favours the chosen response y_{chosen} over the rejected response y_{rejected} . The modification involves the logarithm of the odds ratio, which simplifies optimisation and ensures numerical stability:

$$L_{\text{OR}} = -\log \left(\frac{P(y_{\text{chosen}}|x)}{P(y_{\text{rejected}}|x)} \right)$$

This term penalises the model if the probability of generating y_{chosen} is not significantly higher than the probability of generating y_{rejected} . The overall loss function in ORPO combines the traditional NLL loss with the odds ratio penalty:

$$L_{\text{ORPO}} = L_{\text{SFT}} + \lambda \cdot L_{\text{OR}}$$

4 Experiments

4.1 Dataset

We use the CSEDM 2021 Spring and Fall dataset [16], collected from an introductory Java programming course with 410 students completing 10 assignments

covering foundational concepts like loops and conditionals. Students could submit multiple attempts per task until achieving full marks (100%), with typical solutions ranging from 10 to 20 lines of code.

For this study, we focus on Assignment 1 and construct two datasets from 21,972 submissions: one for SFT and one for RL. The data is split into 18,112 training samples, 1,678 for validation, and 2,182 for testing, following prior work [22] for consistency. **The labelled dataset** (for SFT) pairs each prompt, consisting of the problem (D) and a summary of the student’s past performance (K) with a ground truth solution written by the student (C). **The preference dataset** (for RL) contains the same prompts, but each is paired with a preferred (student-written) and a rejected (LLM-generated) solution. Rejected responses are selected based on a PM Reward System that combines compiler error feedback and CodeBLEU similarity; those with an error score below 0.2 (indicating a weak alignment with student-like error patterns) were selected to ensure pedagogical contrast.

4.2 Baseline Models

We compare the generated codes against two categories of baseline models to evaluate whether they align with the common syntactical errors students will encounter. The first baseline focuses on existing generative systems for personalised code generation. Specifically, we selected the **OKT** [13] model, which generates personalised code based on a student’s knowledge state. While the original OKT implementation used GPT-2, We updated it to use **Llama-3.1-8B**¹, following [3], to ensure fair comparison with modern LLM-based methods. The updated OKT was fine-tuned using our dataset with the same original configurations. We compiled the generated code to evaluate syntactical error alignment and calculated error scores based on the top 10 most common syntax errors observed in student submissions for assignment 1. The second baseline, **Error-DKT** [22], employs a simple LSTM-based Deep Knowledge Tracing (DKT) architecture to directly predict specific errors students are likely to make when submitting their code. This comparison enables us to assess whether simpler models like Error-DKT are more effective at tracing student errors than generative systems that produce personalised code.

4.3 Implementation Details

We experimented two widely adopted instruction-tuned LLMs: **Mistral-7B**² and **CodeLlama-13B**³, both of which have seen increasing use in recent studies on instruction-based language models [26, 27]. Initially, we performed zero-shot in-context learning to assess the models’ capabilities as knowledge tracers and their ability to generate personalised code. In this evaluation, the models were

¹ <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>.

² <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>.

³ <https://huggingface.co/codellama/CodeLlama-13b-Instruct-hf>.

given a prompt consisting of task instructions and student history, without any fine-tuning, to generate code. The generated outputs were compared to ground truth solutions, focusing on their alignment with student-specific knowledge and common syntactical errors. Fine-tuning experiments were conducted across three NVIDIA A80-40G GPUs. We used the Parameter Efficient Fine-Tuning (PEFT) library from HuggingFace and trained the models with Low-Rank Adaptation [6], configured with $\alpha = 64$, $r = 32$, and a dropout rate of 0.05. The training was performed using the AdamW optimiser with a batch size of 64 over three epochs, with a learning rate of 1×10^{-5} , $\lambda = 0.1$ (ORPO). The input prompts were limited to 3,300 tokens, and the maximum output length was set to 250 tokens to align with computational constraints.

4.4 Evaluation

The evaluation process focused on assessing the quality of generated code and its alignment with error patterns observed in actual student submissions. Code similarity was measured using the CodeBLEU metric, which evaluates structural and semantic alignment with ground truth solutions, providing insights into how closely the generated code reflects student submissions. For error classification, the weighted F1 score accounted for class imbalances by fairly evaluating common and rare error types. The weighted F1-beta score prioritised precision over recall, ensuring rare but critical errors were accurately identified for educational interventions. Additionally, the error score, a custom metric (see Sect. 3.3), introduced penalties for misclassifications that result in inconsistencies in pass/fail outcomes, such as predicting a pass when the student would fail, or a fail when the student would pass. In the educational context, the model's utility was assessed in two scenarios: overall performance across all student submissions and performance on first attempts at solving problems. The first scenario provides a broader view of a student's problem-solving process, highlighting the model's ability to align with a student's iterative learning and debugging behaviours. The second scenario focused on early interventions, critical for effective knowledge tracing [4].

5 Results

Our findings demonstrate that, overall, our LLM-based models, fine-tuned for personalised code generation and error prediction, outperform baseline models such as OKT and Error-DKT. Our models generate code solutions that better align with student knowledge and reflect common syntactical errors. Table 1 summarises the results, highlighting substantial improvements in both code similarity and error prediction accuracy.

5.1 Code Generation Task

The code generation task evaluates the model's ability to generate code solutions that align with task requirements and student coding style. Since OKT is the

Table 1. Comparative Performance Analysis of Code Generators across various fine-tuning tasks. This table presents the F1 and F1-beta scores for first and overall submissions and the error score. Additionally, it includes the code blue and the syntax match for the code generation task.

Category	Models	Syntax Error Prediction					Student Code Generation	
		First Attempt		Overall Attempts		Error Score	CodeBleu	Syntax Match
		F1	F1-beta	F1	F1-beta			
Baseline	DKT + Error-DKT	53.1%	53.1%	50.7%	50.1%	78.61%	n/a	n/a
	Error-DKT	50.80%	48.20%	43.30%	41.80%	64.91%	n/a	n/a
	DKT+OKT	39.11%	34.31%	20.35%	16.65%	41.81%	50.16%	57.11%
	OKT	38.73%	33.96%	20.12%	16.46%	41.39%	50.17%	57.11%
Zero-shot	Mistral-7b	40.09%	36.62%	28.11%	28.02%	50.83%	65.72%	66.28%
	Codellama-13b	40.54%	37.82%	34.10%	34.61%	52.82%	67.50%	67.09%
SFT	Mistral-7b	41.60%	39.27%	42.02%	41.64%	58.74%	77.22%	74.84%
	Codellama-13b	40.01%	37.51%	40.61%	40.58%	55.99%	75.49%	73.45%
ORPO	Mistral-7b	42.12%	40.24%	43.36%	43.26%	59.23%	76.57%	74.62%
	Codellama-13b	41.56%	39.94%	41.18%	41.23%	57.48%	75.43%	73.39%
PPO	Mistral-7b	40.13%	37.35%	38.07%	39.63%	56.23%	72.75%	71.65%
	Codellama-13b	40.69%	37.58%	36.74%	37.24%	54.19%	68.60%	68.38%
Two-Step	Codellama	41.66%	39.00%	39.86%	41.19%	62.79%	67.50%	67.09%
	Codellama-SFT	43.40%	41.00%	52.39%	52.68%	73.38%	75.33%	73.20%
	Codellama-ORPO	47.03%	45.38%	56.80%	56.22%	78.77%	75.42%	73.39%
	Codellama-PPO	41.49%	38.49%	43.82%	45.12%	65.74%	68.59%	68.38%
	Mistral	42.33%	38.84%	32.51%	31.91%	58.62%	63.27%	64.76%
	Mistral-SFT	44.89%	42.47%	53.45%	53.31%	74.81%	77.22%	74.84%
	Mistral-ORPO	45.56%	43.65%	55.38%	55.39%	75.99%	76.56%	74.62%
	Mistral-PPO	41.52%	38.81%	43.23%	45.43%	68.74%	61.52%	63.88%

only baseline model designed for code generation, we compare our results against it while excluding Error-DKT, which focuses on error prediction rather than generating student-like code. As a baseline, OKT achieved a CodeBLEU score of 50.16% and a Syntax Match score of 57.11%, showcasing limited capability in capturing student-specific behaviours. Instruction-tuned LLMs like Codellama-13B and Mistral-7B demonstrated significant gains over OKT even in zero-shot settings. Codellama-13B outperformed the baseline with a +17.3% increase in CodeBLEU and a +9.98% improvement in Syntax Match, highlighting the potential of instruction-tuned models as knowledge tracers.

SFT showed improvements in both CodeBLEU and Syntax Match metrics. For example, Code-llama-13B achieved a +25.32% improvement in CodeBLEU and a +16.34% increase in Syntax Match, while Mistral-7B had a +27.05% increase in CodeBLEU and a +17.73% improvement in Syntax Match compared to OKT. SFT consistently improved CodeBLEU and Syntax Match metrics, highlighting its effectiveness in aligning generated code with ground-truth solutions. In contrast, reinforcement learning (RL), including methods like PPO,

did not yield additional improvements over SFT, with CodeBLEU and Syntax Match values either remaining stable or slightly declining. Similarly, the two-step prediction framework incorporating DKT predictions (pass/fail to guide LLM outputs) exhibited similar results. This suggests that SFT is an effective approach for improving overall code similarity and Syntax Match.

5.2 Error Prediction Task

Our primary objective is to predict student-specific errors effectively, leveraging fine-tuned LLMs for improved accuracy. Fine-tuned models, particularly Mistral-7B with RL using ORPO, achieved a +1.46% increase in the overall F1-beta score compared to the best baseline model, Error-DKT, and a +16.59% improvement compared to OKT. Further gains were observed with the two-step prediction framework, where both Codellama-13B and Mistral-7B outperformed the Error-DKT two-step model by +5.29% and +6.12%, respectively, underscoring the effectiveness of this approach.

When comparing first versus overall attempts, both baseline models and non-fine-tuned LLMs exhibited higher prediction rates for first attempts. This is primarily due to the dataset’s bias, where 59.4% of first attempts result in a pass. As a result, models that frequently predict “pass” on first attempts achieve higher scores, reflecting the influence of dataset imbalance rather than an inherent improvement in prediction capability. Fine-tuned LLMs demonstrated superior performance in overall attempts, highlighting their ability to utilise historical information to refine predictions. For instance, Codellama-13B with ORPO achieved a difference of +2.96% between first and overall attempts, while the two-step approach increased this gap to +11.74%, achieving an F1-beta score of 56.22%. These results emphasise the model’s capacity to adapt to complex knowledge-tracing tasks, effectively identifying patterns in iterative learning and student behaviour.

Further analysis of error prediction across individual error categories validates the LLM-based model’s enhancements. Fine-tuned LLMs, especially those using ORPO, consistently improved accuracy across frequent and infrequent error classes. This is particularly valuable in programming education, where rare but impactful errors often indicate critical learning gaps. As shown in Fig. 2, Mistral-7B with ORPO (single and two-step methods) performed well across error categories, including less frequent errors. However, challenges persist for underrepresented classes, such as Class 9, likely due to data imbalance in the training set. Addressing this imbalance could further enhance the model’s ability to predict rare errors effectively.

5.3 Trade-Offs Between Code Similarity and Error Prediction

A key objective of our framework is to ensure that the generated code aligns with students’ problem-solving approaches (structural similarity) while accurately reflecting their likely mistakes (error alignment). Our results suggest that

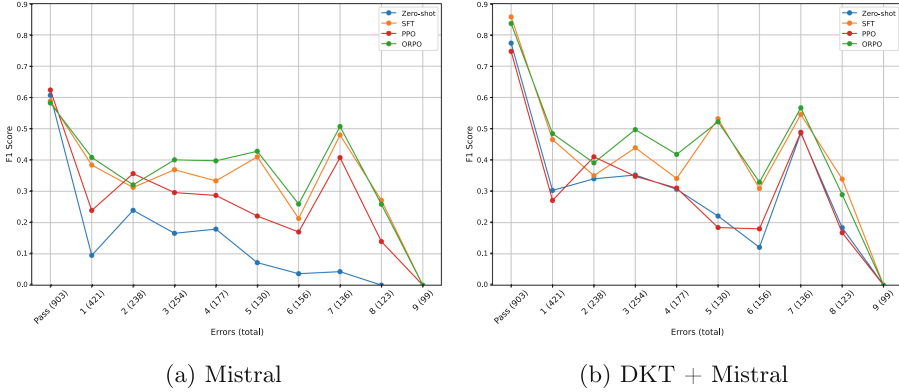


Fig. 2. Error prediction per error class for various models and fine-tuning strategies.

while there is some alignment between code similarity and error prediction, the relationship is not strictly linear.

Fine-tuning (SFT) consistently improves both CodeBLEU and error prediction accuracy. For example, Mistral-7B improves CodeBLEU by +11.50% and the error score by +7.91% compared to its zero-shot counterpart, suggesting that higher structural similarity to student submissions corresponds with improved error prediction. However, reinforcement learning (RL) via ORPO improves error prediction (error score: +0.48%, overall attempts F1-beta: +1.62%) while slightly reducing CodeBLEU by -0.65% compared to its SFT counterpart. This suggests that RL prioritises error alignment over maintaining overall code similarity, refining student-specific mistake patterns without necessarily enhancing coding style. In contrast, PPO leads to a decline in both CodeBLEU and error prediction accuracy, suggesting that improper RL tuning can degrade both syntactic alignment and error consistency.

Although Syntax Match is a component of CodeBLEU, the other factors (N-gram, Weighted N-gram, and DFG) primarily measure broader structural and semantic similarities rather than student-specific error tendencies. This explains why high CodeBLEU scores do not always correlate with better error prediction. For instance, Mistral-7B fine-tuned with SFT achieves a CodeBLEU of 77.22% but underperforms Mistral-7B with ORPO (76.57%) in error prediction. This highlights a limitation in current evaluation methods: most similarity metrics prioritise functional correctness and fluency rather than how well the generated code captures student errors. These findings suggest that models optimised for code similarity do not inherently excel at predicting student-specific errors, meaning the best model for code generation is not necessarily the best for error prediction.

6 Conclusion

We have introduced a framework that integrates LLMs with KT to generate personalised code solutions aligned with students' knowledge states and common error patterns. By fine-tuning open-source LLMs, such as Codellama-13B and Mistral-7B, our approach significantly outperformed baseline models, including OKT and Error-DKT, in both code similarity and error prediction. SFT improved the structural and syntactical alignment of generated code, while RL, particularly ORPO, refined error prediction by capturing student-specific mistake patterns. Additionally, a hybrid approach incorporating DKT predictions enabled a two-step process where LLMs generated either correct or erroneous solutions based on pass/fail predictions. This structured approach ensures that error predictions are student-specific, providing a foundation for more personalised learning support.

Limitations: While these results demonstrate the potential of generative KT models in programming education, several challenges remain. Our study was limited to a few open-source LLMs due to computational constraints, excluding larger models like GPT-4, which may yield even greater improvements. Additionally, the dataset's size and diversity restricted the model's ability to generalise across broader learning scenarios. Expanding the dataset and incorporating a wider variety of programming exercises could enhance scalability and robustness. Furthermore, while RL refined error prediction, it did not consistently improve overall code similarity, highlighting the need for more effective fine-tuning strategies that balance both objectives. Current evaluation metrics also prioritise structural correctness but may not fully capture the complexities of student-specific error modelling, suggesting future work in refining pedagogically relevant evaluation methods.

Future Work: should refine evaluation methodologies to better assess student-like errors and optimise fine-tuning strategies to jointly balance code similarity and error prediction. Our findings suggest that RL reward mechanisms must be carefully designed to ensure that models improve both structural correctness and error alignment without overfitting to specific mistake patterns. Additionally, hybrid evaluation approaches that incorporate both functional correctness and student-specific error tendencies could provide a more comprehensive assessment of model performance. Also, integrating LLM-based error prediction into AIPTs to shift feedback mechanisms from reactive to proactive. By predicting likely errors before students attempt a task, these tools could provide tailored interventions, such as preparatory exercises, targeted hints, or adaptive scaffolding, to better support student learning. This shift toward preemptive support has the potential to enhance personalisation in programming education, making error prediction a valuable foundation for adaptive learning environments.

Acknowledgments. This work was funded by Research Ireland grants 18-/CRT/6223 and 12/RC/2289-P2.

References

1. Altadmri, A., Brown, N.C.: 37 million compilations: investigating novice programming mistakes in large-scale student data. In: SIGCSE 2015 - Proceedings of the 46th ACM Technical Symposium on Computer Science Education, pp. 522–527 (2015). <https://doi.org/10.1145/2676723.2677258>
2. Corbett, A.T., Anderson, J.R.: Knowledge tracing: modeling the acquisition of procedural knowledge. *User Model. User-Adapted Interact.* **4**, 253–278 (1994). <https://doi.org/10.1007/BF01099821/METRICS>
3. Duan, Z., Fernandez, N., Hicks, A., Lan, A.: Test case-informed knowledge tracing for open-ended coding tasks. *arXiv* (2024). <https://doi.org/10.48550/arXiv.2410.10829>
4. Emerson, A., et al.: Predicting early and often: Predictive student modeling for block-based programming environments. In: International Educational Data Mining Society (2019)
5. Hong, J., Lee, N., Thorne, J.: ORPO: monolithic preference optimization without reference model. *arXiv* (2024). <http://arxiv.org/abs/2403.07691>
6. Hu, E.J., et al.: LORA: low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021)
7. Ibarz, B., Leike, J., Pohlen, T., Irving, G., Legg, S., Amodei, D.: Reward learning from human preferences and demonstrations in Atari. In: Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc. (2018)
8. Kim, J., Chu, S., Wong, B., Yi, M.: Beyond right and wrong: mitigating cold start in knowledge tracing using large language model and option weight. *arXiv preprint arXiv:2410.12872* (2024)
9. Le, H., Wang, Y., Gotmare, A.D., Savarese, S., Hoi, S.C.: CODERL: mastering code generation through pretrained models and deep reinforcement learning. In: *Advances in Neural Information Processing Systems*, vol. 35 (2022). <https://arxiv.org/abs/2207.01780v3>
10. Lee, U., et al.: From prediction to application: language model-based code knowledge tracing with domain adaptive pre-training and automatic feedback system with pedagogical prompting for comprehensive programming education. *arXiv* (2024). <https://doi.org/10.13140/RG.2.2.25134.11847>
11. Liang, Y., Peng, T., Pu, Y., Wu, W.: HELP-DKT: an interpretable cognitive model of how students learn programming based on deep knowledge tracing. *Sci. Rep.* **12**(1), 4012 (2022). <https://doi.org/10.1038/s41598-022-07956-0>
12. Liu, J., et al.: RLTF: reinforcement learning from unit test feedback. *ArXiv* (2023). <https://arxiv.org/abs/2307.04349v2>
13. Liu, N., Wang, Z., Baraniuk, R., Lan, A.: Open-ended knowledge tracing for computer science education (2022) <https://aclanthology.org/2022.emnlp-main.254>
14. Medeiros, R.P., Ramalho, G.L., Falcao, T.P.: A systematic literature review on teaching and learning introductory programming in higher education. *IEEE Trans. Educ.* **62** (2019). <https://doi.org/10.1109/TE.2018.2864133>
15. Meliana, S., Nurjanah, D.: Adopting good-learners' paths in an intelligent tutoring system. In: 2018 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE), pp. 877–882 (2018). <https://doi.org/10.1109/TALE.2018.8615176>
16. Organizers, C.: The 2nd CSEDM data challenge. <https://sites.google.com/ncsu.edu/csedm-dc-2021/> (2021). Accessed 1 Mar 2022

17. Ouyang, L., et al.: Training language models to follow instructions with human feedback. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS 2022, Curran Associates Inc., Red Hook, NY, USA (2022)
18. Phung, T., et al.: Generating high-precision feedback for programming syntax errors using large language models. In: Proceedings of the 16th International Conference on Educational Data Mining, vol. abs/2302.04662, pp. 370–377 (2023). <https://doi.org/10.48550/arXiv.2302.04662>, <https://www.semanticscholar.org/paper/a525cbb8600ab6d2a1351f6883937c1774642d76>
19. Piech, C., et al.: Deep knowledge tracing. In: Advances in Neural Information Processing Systems, vol. 28 (2015)
20. Ren, S., et al.: CodeBLEU: a method for automatic evaluation of code synthesis. arXiv (2020). <https://arxiv.org/abs/2009.10297v2>
21. Rivers, K., Harpstead, E., Koedinger, K.: Learning curve analysis for programming: which concepts do students struggle with? In: Proceedings of the 2016 ACM Conference on International Computing Education Research, pp. 143–151. ICER 2016, Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2960310.2960333>
22. Shaka, M., Carraro, D., Brown, K.N.: Error tracing in programming: a path to personalised feedback. In: Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024), pp. 330–342. Association for Computational Linguistics, Mexico City, Mexico (2024). <https://doi.org/10.18653/v1/2024.bea-1.27>, <https://aclanthology.org/2024.bea-1.27>
23. Shen, B., et al.: PANGU-Coder2: boosting large language models for code with ranking feedback. arXiv (2023). <https://arxiv.org/abs/2307.14936v1>
24. Shi, Y., Chi, M., Barnes, T., Price, T.W.: Code-DKT: a code-based knowledge tracing model for programming tasks. arXiv (2022)
25. Shojaee, P., Jain, A., Tipirneni, S., Reddy, C.K.: Execution-based code generation using deep reinforcement learning. Trans. Mach. Learn. Res. (2023). <https://github.com/reddy-lab-code-research/PPOCoder>
26. Tai, I.C., Wong, E.C., Wu, J.T., Leung, K., Yap, D.Y., Wong, Z.S.: Exploring offline large language models for clinical information extraction: a study of renal histopathological reports of lupus nephritis patients. Stud. Health Technol. Inf. **316**, 899–903 (2024). <https://doi.org/10.3233/SHTI240557>, <https://pubmed.ncbi.nlm.nih.gov/39176938/>
27. Thakkar, H., Manimaran, A.: Comprehensive examination of instruction-based language models: a comparative analysis of mistral-7b and llama-2-7B. In: 2023 International Conference on Emerging Research in Computational Science (ICERCS), pp. 1–6 (2023). <https://doi.org/10.1109/ICERCS57948.2023.10434081>
28. Wang, S., Han, Y., Wu, W., Hu, Z.: Modeling student learning outcomes in studying programming language course. In: 2017 Seventh International Conference on Information Science and Technology (ICIST), pp. 263–270 (2017). <https://doi.org/10.1109/ICIST.2017.7926768>
29. Wei, J., et al.: Finetuned language models are zero-shot learners. In: International Conference on Learning Representations (2022)